

Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	29 June 2026
Project Name	Sports hall booking system
Team ID	

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

Reference: <https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processingduring-pandemics/>

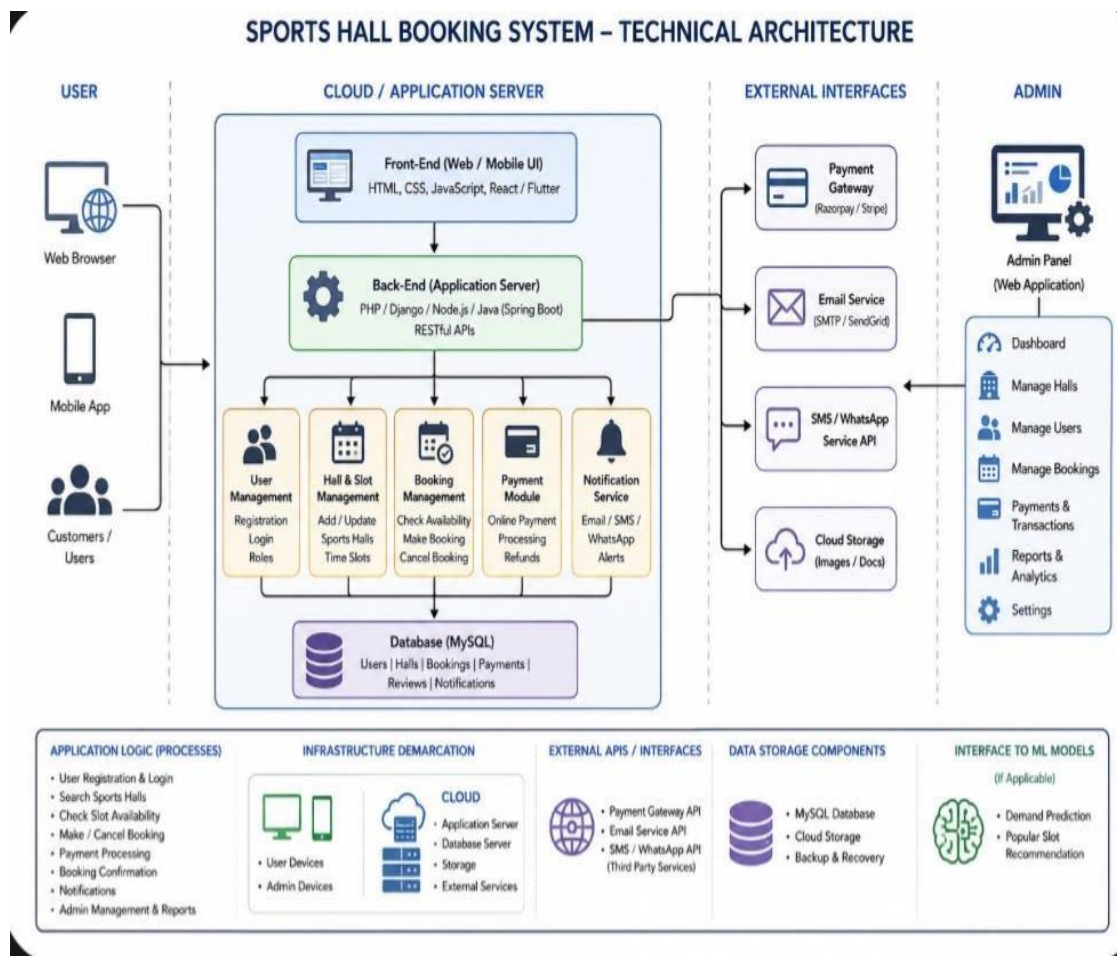


Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	How users interact with application e.g. Web (Browser), Mobile App etc.	HTML, CSS, JavaScript / React / Angular / Bootstrap, Flutter
2	Application Logic – 1	Logic for user authentication and authorization	Java / Python (Django) / Node.js
3	Application Logic – 2	Logic for managing sports halls, time slots and availability	Java / Python (Django) / Node.js
4	Application Logic – 3	Logic for booking creation, modification and cancellation	Java / Python (Django) / Node.js
5	Database	Stores all system data (users, halls, bookings, payments, etc.)	MySQL / PostgreSQL
6	Cloud Database	Managed database service on cloud	AWS RDS / Google Cloud SQL / Azure SQL
7	File Storage	Stores images (hall photos, documents, etc.)	AWS S3 / Google Cloud Storage
8	External API – 1	Payment gateway API for processing payments	Razorpay / Stripe / PayPal API
9	External API – 2	Email / SMS API for notifications	SendGrid / Twilio / MSG91 API
10	Reporting & Analytics	Generate reports on bookings, revenue, user activity, etc.	Google Data Studio / Custom Reports
11	Infrastructure (Server / Cloud)	Application deployment on local or cloud infrastructure	AWS / Azure / Google Cloud / Local Server

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	List the open-source frameworks used	Django / Spring Boot / React / Flutter
2	Security Implementations	List all the security / access controls implemented, use of firewalls, encryption, etc.	JWT Authentication, HTTPS, Role-based Access Control, Firewall, OWASP

Table-3: Non-Functional Characteristics

S.No	Characteristics	Description	Technology
1	Scalable Architecture	Justify the scalability of architecture (3 – tier / microservices)	3-Tier Architecture / Microservices
2	Availability	Justify the availability of application (e.g. use of load balancers, distributed servers etc.)	Load Balancer, Auto Scaling, Database Replication
3	Performance	Design consideration for the performance of the application (number of requests per sec, caching, CDN, etc.)	Caching (Redis), CDN, Optimized Queries, Performance Monitoring
4	Reliability	Mechanisms to ensure reliability and fault tolerance	Database Backup, Failover, Redundancy
5	Maintainability	How easy it is to maintain and update the system	Modular Code, Version Control (Git), CI/CD Pipeline

References:

- <https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/>
- <https://aws.amazon.com/architecture/>
- <https://docs.microsoft.com/en-us/azure/architecture/>
- <https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Deployment>
- <https://owasp.org/www-project-top-ten/>